Introduction to Data Centre Management Using a Simulation Game

José M. A. M. Santos[1] and Frutuoso G. M. Silva[1,2][0000−0002−0611−6157]

[1] University of Beira Interior, Covilhã, Portugal
[2] Instituto de Telecomunicações, Covilhã branch, Portugal
{jose.melo.santos,fgmsilva}@ubi.pt

Abstract. This paper presents the design, development and exploratory evaluation of Port 0, a resource management video game in which the player operates a small data centre in a science-fiction setting. The game combines server hardware configuration, a typing-based command minigame and a periodic defence sequence that represents a DDoS attack, with the intention of exposing players to data centre terminology and concepts through gameplay. A single-group pre/post study was conducted remotely with 25 participants of mixed technical backgrounds, who played for about 19 minutes. Recognition increased for most terms that appear in the game, such as the ARM architecture, IBM as a CPU vendor, software installation through a terminal and DDoS attacks, whereas options not covered by the game, such as ransomware, remained unchanged. Given the small sample, the short exposure, the immediate post-test and the absence of a comparison group, these results are preliminary indications of term recognition and not evidence of management skills. Usability observations analysed with Nielsen's heuristics [26] and telemetry collected with Firebase informed interface changes. The results suggest that mechanics-driven simulation games are a promising way to introduce data centre vocabulary and justify controlled studies.

Keywords: Data centers simulation · Management in data centers · Simulation games · Game-Based Learning.

1 Introduction

The video game industry has grown substantially over the past decade and, as it expands in scale and relevance, the application of video games increasingly extends beyond entertainment.

Serious games and simulation-based games provide players with interactive experiences that incorporate educational content. However, even games developed exclusively for entertainment require players to acquire new knowledge, develop complex skills and devise effective strategies in order to progress [2]. Similarly, Miller [1] suggests that the content of video games can facilitate indirect learning, that is, learning that occurs without the game being explicitly designed for educational purposes.

Motivated by these findings, this work investigates whether players acquire knowledge while interacting with **Port 0**, a resource management video game in which the player operates a small data centre within a fictional setting. Players must purchase and configure servers, fulfil client requests and defend the infrastructure against cyberattacks. Although the game is set in a science fiction universe, it incorporates simulation-based mechanics, including a word-based minigame and server configuration tasks.

We defined two levels of learning objectives (LO, i.e., the knowledge that players are expected to acquire by playing): **LO1**, recognition of terms related to server hardware, CPU architectures and vendors, operating systems, server software and cyberattacks; and **LO2**, conceptual understanding, e.g., that server components determine performance and energy consumption, that software can be installed through a terminal and that a DDoS attack disrupts service availability. This exploratory study addresses the following research questions (RQ, i.e., the questions that the evaluation seeks to answer):

- **RQ1**: Does a short session of Port 0 change participants' recognition of the data centre terms embedded in its mechanics?
- **RQ2**: Are such changes specific to content present in the game, when compared with content that the game does not address?
- **RQ3**: Which usability problems hinder the first contact with the game?

The evaluation mainly measures LO1 and, partially, LO2.

2 Related Work

This section reviews the learning strategies that Port 0 draws on, related games about IT infrastructure and security, and the public interest in the game's theme.

2.1 Learning through observation and through interaction

Observational learning is the process by which beginners absorb rules, initial steps and complex procedures by watching experienced players, simulations or tutorials [2]. When integrated into video games or simulations, it acts as a pedagogical "scaffold" for the learner [2]. Titles such as CodeCombat [12], Manufactoria [13] and Labor Support Game [14] employ this strategy. In CodeCombat, for example, the code is highlighted line by line while the hero performs the corresponding action, so players directly observe the outcome of each command [3].

Observing a demonstration differs from learning through interaction, in which learners act on the system and receive feedback on the consequences of their actions [3,6]. Port 0 relies mainly on interaction and feedback, but its WordRush minigame also includes an observational component similar to CodeCombat: each completed command is shown in a history panel together with a short description of its effect (Section 3.2).

2.2 Trial and error and self-directed learning

Trial and error is an active and exploratory strategy in which individuals test hypotheses in real time to determine which actions lead to success or failure [3]. Learners repeatedly attempt tasks until they succeed or reach the highest score, understanding complex concepts through their own mistakes [4,5]. Feedback loops reinforce this technique, as the system continuously responds to the player's actions [6]. Port 0 incorporates gameplay loops in which the player repeats a set of tasks and receives feedback in the form of payments, energy costs and attacks (Section 3).

A related concept is self-directed learning, in which individuals take the initiative to diagnose their learning needs, choose resources and evaluate the outcomes [7]. It is common in fields such as programming, where solutions are developed by continuously testing what works [8].

2.3 Indirect learning

Indirect learning is a process in which knowledge is acquired as a by-product of another activity, such as leisure, often without the individual realising that learning is taking place [9,8]. Rather than explicitly teaching content, games create environments in which knowledge is constructed through experience, problem-solving and interaction with complex systems [1]. Commercial games such as *No Man's Sky* [10] and *The Legend of Zelda: Breath of the Wild* [11] promote experimentation, creativity, complex problem-solving and resource management; according to Miller, *No Man's Sky* encourages behaviours similar to those used in scientific inquiry without explicitly teaching science [1].

Following this idea, Port 0 places technical terms in its environment, interface and tasks, so that players can encounter them while playing. In this paper, direct and indirect learning are treated as *design intentions*: the evaluation does not distinguish between them.

2.4 Games about IT infrastructure and security

Several commercial games represent IT infrastructure. *Data Center* [15] focuses on the physical layout of racks, power and cooling with high technical fidelity; *PC Building Simulator* [16] reproduces the assembly of a computer and the installation of its operating system; *Startup Company* [17] and *Software Inc.* [18] combine business management with server, power and cooling constraints; and *Grey Hack* [19] simulates networks and hacking through command-line terminals. These titles are designed for entertainment and, to our knowledge, have not been evaluated as learning tools. In contrast, CyberCIEGE [20] is a serious game in which players configure and protect a virtual network under budget constraints, and it has been used in information assurance education. More broadly, reviews of empirical studies associate games with knowledge acquisition and motivation, but also note that many studies rely on small samples and weak designs [21].

Port 0 lies between these groups: it simplifies the infrastructure represented in commercial simulators and adds fantasy elements, following design guidelines for educational games [22], and it is evaluated, here in an exploratory way, for the recognition of the concepts it embeds.

2.5 Public interest in the technology field

Google Trends reports the relative search interest (0–100) for terms over time. Figure 1 shows the worldwide interest for the search terms *technology* and *data center* from August 2021 to August 2026 (monthly values, retrieved in August 2026). Interest in both terms remained roughly stable until mid-2025 and then rose sharply. Since the context of a game can raise a player's interest before play begins [2] and the attractiveness of an educational game is related to its subject matter [4], we hypothesise that this growing public attention may increase learners' interest in the topic. This hypothesis is not tested in the present study.

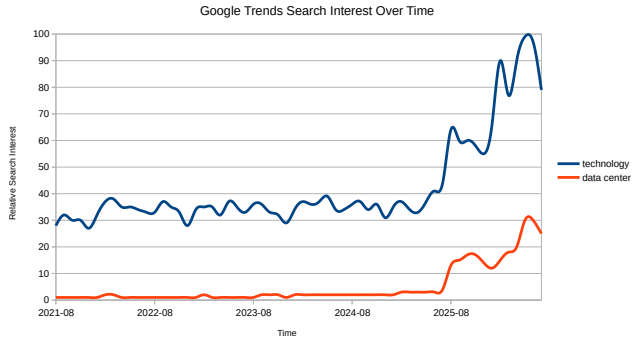


Fig. 1. Worldwide relative search interest for the terms *technology* and *data center* (Google Trends, August 2021–August 2026, retrieved in August 2026).

3 Design and development of Port 0 game

3.1 Design of Port 0 game

Port 0 is set in a fantasy environment with science-fiction elements in which the player is an engineer working in a facility inspired by a data centre. The main tasks are to buy and configure servers, taking into account aspects such as processing capacity and energy consumption, and to complete client requests through a word-based minigame inspired by entering commands in a terminal. In addition, the player must defend the data centre from attacks using a cannon.

The game features two main gameplay loops. The task loop is the most common and longest-lasting: the player receives client requests and chooses whether

to accept them; an accepted request is completed by playing the WordRush minigame. In the attack loop, the player uses a cannon to defend the data centre from hordes of enemies. Table 1 maps the content of the learning objectives to the mechanics that convey it and to the questionnaire items used in the evaluation (Section 4).

Table 1. Mapping between learning content, game mechanics and assessment items.

Content (LO)	**Game mechanic**	**Questionnaire item**
Server hardware (LO1, LO2)	Shop: configuration of CPU, RAM, disks, network card, OS and simplified cooling, with short descriptions	Components heard of
CPU architectures and vendors (LO1)	Shop: choice of ARM/x64 and AMD/Intel/IBM; ARM recommended when energy use is high	CPU architectures; other CPU vendors
Operating systems (LO1)	OS installation step in the shop; Ubuntu-styled terminal in WordRush	Other operating systems
Data centre services and server software (LO1, LO2)	Client requests; WordRush categories (Apache, MySQL, Node.js, Nginx, Postfix)	Data centre services; website technologies; e-mail service
Software installation via terminal (LO1, LO2)	WordRush commands (e.g., `sudo apt install`)	Installation methods
Cyberattacks: DDoS (LO1, LO2)	Attack loop: revenue suspended until all enemies are destroyed	Cyberattacks
Not addressed by the game	–	Ransomware; DVD, app store and website installation

Three game design techniques guided this concept: the MDA framework [23], Flow [24] and the Hook Model [25]. In MDA terms, the *mechanics* are typing Linux commands in WordRush, configuring server hardware and operating the defence cannon; they give rise to *dynamics* of resource management, balancing task revenue against energy expenses and adapting to attack interruptions; and the intended *aesthetics* are a sense of technical mastery, with task monotony broken by action-oriented combat.

Flow [24] describes a balance between challenge and the player's abilities that avoids boredom and anxiety. It was used as a design reference: the attack loop was introduced to interrupt the potentially monotonous task loop and renew the challenge. Flow and engagement were design goals and were not measured in this study.

The Hook Model [25] describes a cycle of trigger, action, variable reward and investment intended to form habits. Because habit-forming design raises ethical concerns, particularly for young players, its use in Port 0 was limited to elements that sustain a play session: curiosity about the theme, short tasks of about one minute and variable payments from clients. The prototype has no monetisation, and planned reward elements such as trophies were not implemented. Client revenue continues to accrue while the game is closed, which acts as a return trigger; its effect on long-term play was not studied, as the evaluation consisted of a single supervised session.

3.2 Development of Port 0 game

Port 0 was developed with *Unity* and *C#*, and the 3D models were created and textured in *Blender*. The game is composed of several interconnected systems (Fig. 2).

The **shop system** allows the player to purchase a server through a sequence of steps in which components are configured, each with a short description (Fig. 3). Every component has a price and an energy consumption in watts, and processors contribute to computing capacity in TFLOPS. Real units are used to connect the simulation with real hardware, although values are simplified. When the configured server has a high energy consumption, the shop displays a message recommending an ARM processor, reflecting the common positioning of ARM server processors as more energy-efficient. After the hardware, the player goes through a simulated operating system installation, in which the server is named.

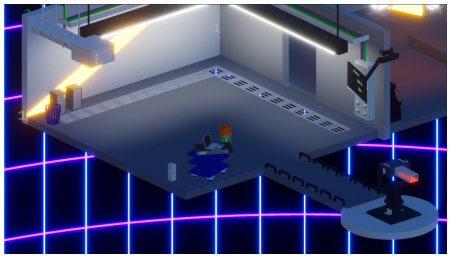


Fig. 2. Game scene.

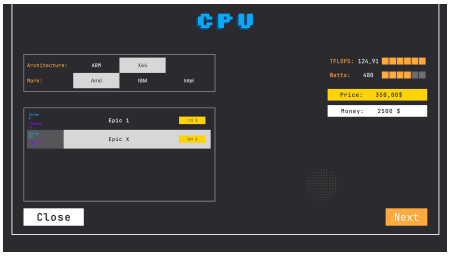


Fig. 3. Shop: CPU configuration, with architecture, vendor, TFLOPS, watts and price.

The **economy system** periodically debits energy costs and credits one-off and recurring payments from clients; cooling, water and network costs were omitted in this version. The **task system** periodically presents client requests, each requiring server resources, which the player may accept or reject. The player earns money actively, by completing new tasks, and passively, through recurring payments from previously completed ones.

An accepted task is completed by playing **WordRush** (Fig. 4), which simulates an Ubuntu-styled terminal. The words of a command (e.g., `sudo`, `apt`, `install`, `mysqlserver`) move across the screen and the player must type them as displayed; players do not choose or compose the commands. Each completed command is added to a history panel together with a short description of its effect (e.g., "Installs MySQL server"). Each task category corresponds to a server technology: Apache, MySQL, Node.js, Node.js with Nginx, and Postfix. WordRush therefore provides exposure to commands and their purpose, not practice in writing them.



Fig. 4. WordRush minigame: words of the current command move across the terminal (right); completed commands and their descriptions are listed in the history panel (left).

Finally, the **combat system** triggers an attack after a given number of completed tasks. The attack is announced by an e-mail and, while it lasts, recurring revenue is suspended but costs continue, until the player destroys all enemies with the cannon. The cannon is not intended to represent real DDoS mitigation, such as traffic filtering or rate limiting. It is a metaphor meant to convey that a data centre can be targeted and that an attack disrupts service availability and causes financial loss; its main design purpose is to break the monotony of the task loop.

4 Evaluation Method

4.1 Participants

Twenty-five participants were recruited by convenience among friends, acquaintances and through announcements at the university. Their ages ranged from under 18 to 34 years, with 21 aged between 18 and 29; two participants were under 18 (Table 2). Thirteen participants had an academic background in computer science (Table 3); one answer to this question was invalid and was excluded, so Table 3 totals 24.

4.2 Procedure

Sessions were conducted remotely through Discord or Google Meet, with participants sharing their screen. Each participant answered the pre-test questionnaire, played until the end of the first DDoS attack (on average 19 minutes, Table 6) and answered the post-test immediately afterwards. The researcher intervened

Table 2. Age range.

Age range	Participants
Under 18 years old	2
18 to 24 years old	10
25 to 29 years old	11
30 to 34 years old	2
35 years old or over	0

Table 3. Prior knowledge in computer science ($n = 24$ valid answers).

Prior knowledge	Participants
Academic training in the area	13
User level only	4
None	7

only when a participant could not progress for more than two minutes. The study was approved by the ethics committee, participation was voluntary, the questionnaires were anonymous, and no personal data were collected nor profiles built.

4.3 Instruments

The pre-test included demographic questions (age, education, training in computer science and gaming habits) and nine knowledge items, which were repeated unchanged in the post-test. Six items were multiple-response questions, e.g., "Which of the following are cyberattacks?", mixing correct options, incorrect options and fictitious distractors; one was a yes/no question ("Can an individual create a Gmail-like e-mail service?"); and two asked which CPU vendors other than AMD and Intel, and which operating systems other than Windows and macOS, participants knew (open answers). For each option we report the number of participants who selected it before and after playing. Several options correspond to content that is not present in the game and act as a comparison for the effect of repeating the test. Because the two anonymous questionnaires were not linked at participant level, paired tests (e.g., exact McNemar) could not be computed, and the results are descriptive.

For the usability analysis, the researcher observed each shared screen and recorded the problems encountered, which were afterwards classified according to Nielsen's heuristics [26]. Severity was assigned as *low* (no direct impact on gameplay), *medium* (reduced impact, without preventing progress) or *high* (strong impact, may block progress).

Firebase recorded timestamps of gameplay events, from which we computed the time to purchase and configure the first server, the duration of the first attack (from its start until the last enemy is destroyed), the total time from the start of the session until the end of the first attack, and the time to complete each WordRush task. Six of the 25 telemetry records were discarded as invalid, due to game errors or external factors, leaving 19 records.

5 Results

5.1 Knowledge items

Table 4 reports all questionnaire options. Percentage changes are given in percentage points (pp).

Table 4. Number (and percentage) of the 25 participants who selected each option before and after playing. Key: C = correct, I = incorrect or fictitious. In game: whether the content appears in Port 0.

Item	Option	Key	In game	Before	After	Δ (pp)
Components heard of	CPU	C	yes	23 (92%)	25 (100%)	+8
	Disks	C	yes	23 (92%)	25 (100%)	+8
	Operating system	C	yes	23 (92%)	23 (92%)	0
	Fans	C	partly	23 (92%)	23 (92%)	0
	Motherboard	C	no	23 (92%)	23 (92%)	0
CPU architectures	x64	C	yes	22 (88%)	25 (100%)	+12
	ARM	C	yes	14 (56%)	21 (84%)	+28
	x32[a]	C	no	20 (80%)	18 (72%)	−8
	x8	I	no	2 (8%)	1 (4%)	−4
Other CPU vendor	IBM	C	yes	2 (8%)	12 (48%)	+40
Other OS	Linux	C	yes	19 (76%)	22 (88%)	+12
Data centre services	Website hosting	C	yes	15 (60%)	22 (88%)	+28
	Data storage	C	yes	19 (76%)	22 (88%)	+12
	E-mail, DNS, VPN	C	yes	15 (60%)	21 (84%)	+24
Website technologies	Programming languages	C	partly	25 (100%)	24 (96%)	−4
	“Apache framework”[b]	C	yes	12 (48%)	18 (72%)	+24
	Windows	I	–	0 (0%)	2 (8%)	+8
	Gmail	I	–	0 (0%)	1 (4%)	+4
	Microsoft Word	I	–	0 (0%)	0 (0%)	0
Individual e-mail service	Yes	C	yes	19 (76%)	22 (88%)	+12
Installation methods	Terminal commands	C	yes	15 (60%)	24 (96%)	+36
	Download from website	C	no	18 (72%)	20 (80%)	+8
	App store	C	no	16 (64%)	19 (76%)	+12
	DVD	C	no	21 (84%)	20 (80%)	−4
	Asking an AI	I	–	5 (20%)	5 (20%)	0
	“Power of the mind”	I	–	1 (4%)	0 (0%)	−4
Cyberattacks	DDoS	C	yes	17 (68%)	24 (96%)	+28
	Ransomware	C	no	18 (72%)	18 (72%)	0
	“Wifi”	I	–	2 (8%)	0 (0%)	−8
	“BNS”	I	–	1 (4%)	0 (0%)	−4

[a] x32 is an ABI of the x86-64 architecture rather than an architecture. [b] Wording of the instrument; Apache HTTP Server is a web server, not a framework (see text).

The largest increases occurred in options directly present in the game: IBM as a CPU vendor (+40 pp), which is one of the three vendors in the shop; installation through terminal commands (+36 pp), the only installation method shown in WordRush; the ARM architecture (+28 pp), which the shop recommends for lower energy consumption; DDoS (+28 pp); and website hosting (+28 pp). In contrast, ransomware, which the game does not address, remained unchanged (18 before and after), as did "asking an AI"; installation by DVD and the x32 option, also absent from the game, decreased slightly. Recognition of hardware components was already near the ceiling before playing. Two participants selected incorrect website technologies after playing.

The option "Apache framework, among others" was worded incorrectly in the instrument: Apache HTTP Server is a web server, not a framework. We therefore interpret this item as recognition of server software required to host a website. Since the option was scored as correct in both questionnaires, the wording affects the interpretation of the item but not the comparison between the two moments.

5.2 Usability

Table 5 lists the problems observed during the sessions. All participants played the same version of the game; the proposed solutions were implemented after the sessions and have not yet been evaluated with users.

Table 5. Usability problems classified with Nielsen's heuristics.

Problem	Severity	Heuristic	Solution
Lack of feedback in the sound system	Low	Visibility of system status	Informative label
Lack of feedback in the notification buttons	Medium	Visibility of system status	Clearer instructions next to the buttons
Lack of feedback in the CPU configuration (shop)	Medium	Visibility of system status	Informative label
Cannon controls	High	User control and freedom / Consistency and standards	Inverted controls
Player does not remember how to open the shop	High	Recognition rather than recall	Visual indicator on the UI
Player unaware of the purpose of hardware components	High	Match between system and the real world	Short component descriptions in the shop
Lack of feedback for e-mails	High	Visibility of system status	Indicator shown whenever there is an unread e-mail
Camera too close to the player	Low	User control and freedom	Camera zoom control

5.3 Telemetry

Table 6 summarises the times computed from the 19 valid telemetry records.

Table 6. Descriptive statistics of the main times ($n = 19$ valid records). M: mean; SD: standard deviation; CV: coefficient of variation.

Metric	n	M (s)	SD (s)	CV (%)
Purchase of the first server	19	125.94	84.88	67.39
Duration of the first attack	19	85.33	25.11	29.42
Total time until the end of the first attack	19	1144.90	411.01	35.90

The purchase of the first server shows the highest variability (CV of 67.39%, Fig. 5). One possible explanation is the heterogeneity of prior computer science experience in the sample, but no subgroup analysis was performed and this remains a hypothesis. The duration of the first attack shows lower variability (CV of 29.42%), which may reflect that the cannon mechanic resembles those of many other games; lower variance does not, by itself, indicate learning. The total time until the end of the first attack (M $\approx$ 19 min, CV of 35.90%) varied considerably between participants.

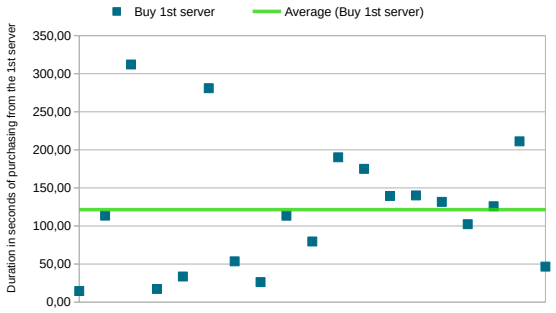


Fig. 5. Time required by each participant to purchase the first server ($n = 19$).

The mean time per WordRush category ranged from 31 to 79 seconds (Fig. 6). This is consistent with the design target of short tasks of about one minute (Section 3), which was chosen to avoid keeping the player in a single activity for too long while allowing some variation between tasks.

6 Discussion and Limitations

Regarding RQ1, recognition increased for most terms embedded in the game's mechanics. Regarding RQ2, options absent from the game, such as ransomware, did not show a comparable increase, which is consistent with an effect of exposure to the game content rather than a generic improvement. Regarding RQ3, the most severe problems concerned feature discoverability (opening the shop), feedback (e-mails) and the cannon controls.

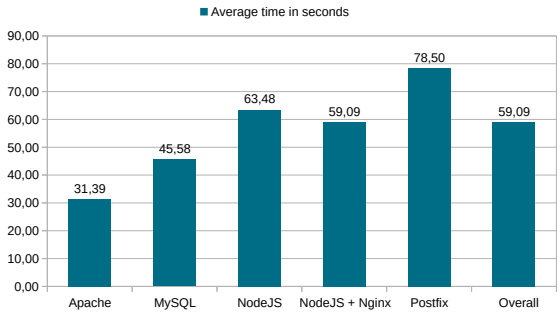


Fig. 6. Mean time per task category in the WordRush minigame.

These results must be interpreted with caution. The study used a small convenience sample, a single short session (about 19 minutes) and an immediate post-test, with no comparison group and no retention test. A single-group pre/post design cannot separate the effect of playing from that of repeating the same questionnaire, and a few unchanged items do not establish causation. Participants received help when blocked for more than two minutes, the sessions were remote, and the analysis is descriptive, as the questionnaires could not be paired. The instrument measures recognition: selecting "terminal commands" does not show that a participant can install software or explain a command, and recognising DDoS does not show knowledge of how to mitigate it; management decisions were not assessed. Finally, one item used incorrect terminology (Apache), and the technical content would benefit from review by data centre professionals.

7 Conclusion

This paper presented the design, development and exploratory evaluation of **Port 0**, a resource management simulation game designed to expose players to data centre concepts through server configuration, a command-typing minigame and a periodic attack loop. In a study with 25 participants, recognition increased for most terms embedded in the gameplay, such as the ARM architecture, IBM as a CPU vendor, terminal-based installation and DDoS attacks, whereas content not addressed by the game showed no comparable change. Usability observations and telemetry identified interface problems that guided subsequent improvements.

Given the limitations of the study, these findings are preliminary indications that mechanics-driven simulation games can introduce technical vocabulary, not evidence that players acquire data centre management skills. Future work will expand the operational challenges of Port 0, introduce further security scenarios such as ransomware, include items that assess conceptual understanding and management decisions, re-evaluate the usability changes and conduct controlled studies with larger samples and delayed post-tests to assess retention.

Acknowledgment. This work is funded by national funds through FCT — Fundação para a Ciência e a Tecnologia, I.P., and, when eligible, co-funded by EU funds under project/support UID/50008/2025 — Instituto de Telecomunicações.

Disclosure of Interests. The authors have no competing interests to declare that are relevant to the content of this article.

References

1. Miller, M.: Video Games and Indirect Learning: A Study of 12 Games That Can Teach. In: Esports Research and Its Integration in Education, pp. 69–84. IGI Global (2021)
2. Martens, A., Diener, H., Malo, S.: *Game-Based Learning with Computers Learning, Simulations, and Games*, vol. 5080, Berlin, Heidelberg (2008)
3. Villareale, J., Biemer, C. F., Seif El-Nasr, M., & Zhu, J. *Reflection in Game-Based Learning: A Survey of Programming Games*. In *Proceedings of the International Conference on the Foundations of Digital Games (FDG '20)*. ACM, 2020. doi:10.1145/3402942.3403011
4. Marques, B. P., Reis, R., Cardoso, M.: *Games: The Motivation in Engineering Education*. In: Ninth International Conference on Technological Ecosystems for Enhancing Multiculturality (TEEM'21), pp. 395–399. ACM, New York (2021). doi:10.1145/3486011.3486483
5. Sari, N. M., Yaniawati, P., Supianti, I. I., Indriani, R.: *Digital Game-Based Learning Interventions on Students' Numeracy Skills and Engagement*. Formatif: Jurnal Ilmiah Pendidikan MIPA, 15(1), 39–50 (2025). doi:10.30998/formatif.v15i1.23356
6. Ibrahim, R., Jaafar, A.: *Using Educational Games in Learning Introductory Programming: A Pilot Study on Students' Perceptions*. In: IEEE International Conference, pp. 1–5. IEEE (2010). doi:10.1109/ITSIM.2010.5561324
7. Knowles, M.S.: *Self-Directed Learning: A Guide for Learners and Teachers*. Association Press, New York (1975)
8. Baxter-Webb, J.: *The Role of Gaming Platforms in Young Males' Trajectories of Technical Expertise*. Transactions of the Digital Games Research Association (ToDiGRA), 2(3), 89–115 (2016). doi:10.26503/todigra.v2i3.58
9. Lindberg, R. S. N., Laine, T. H., Haaranen, L.: *Gamifying Programming Education in K-12: A Review of Programming Curricula in Seven Countries and Programming Games*. British Journal of Educational Technology, 50(4), 1979–1995 (2019). doi:10.1111/bjet.12685
10. Hello Games: *No Man's Sky*. Video game. Hello Games, Guildford (2016)
11. Nintendo EPD: *The Legend of Zelda: Breath of the Wild*. Video game. Nintendo, Kyoto (2017)
12. CodeCombat Inc.: *CodeCombat*. Online educational game, `https://codecombat.com/`, last accessed 2026/09/03
13. PleasingFungus Games: *Manufactoria 2022*. Video game, Steam, `https://store.steampowered.com/app/1276070/Manufactoria_2022/`, last accessed 2026/09/03
14. Chang, C.-Y., et al.: *Labor Support Game*: Scenario game-based learning system for nursing training. Interactive Learning Environments (2024). https://doi.org/10.1080/10494820.2024.2308092
15. Waseku: *Data Center*. Video game, Steam (2026), `https://store.steampowered.com/app/4376050/Data_Center_Demo/`

16. Claudiu Kiss: *PC Building Simulator*. Video game, The Irregular Corporation (2019), https://store.steampowered.com/app/621060/PC_Building_Simulator/
17. Hovgaard Games: *Startup Company*. Video game, Steam (2020), https://store.steampowered.com/app/606800/Startup_Company/
18. Coredumping: *Software Inc.* Video game, Steam (2015), https://store.steampowered.com/app/362620/Software_Inc/
19. Loading Home: *Grey Hack*. Video game, Steam (2017), https://store.steampowered.com/app/605230/Grey_Hack/
20. Irvine, C.E., Thompson, M.F., Allen, K.: *CyberCIEGE: Gaming for Information Assurance*. IEEE Security & Privacy **3**(3), 61–64 (2005). https://doi.org/10.1109/MSP.2005.64
21. Connolly, T.M., Boyle, E.A., MacArthur, E., Hainey, T., Boyle, J.M.: *A Systematic Literature Review of Empirical Evidence on Computer Games and Serious Games*. Computers & Education **59**(2), 661–686 (2012). https://doi.org/10.1016/j.compedu.2012.03.004
22. Silva, F.G.M.: *Practical Methodology for the Design of Educational Serious Games*. Information **11**(1), 14 (2020). https://doi.org/10.3390/info11010014
23. Hunicke, R., LeBlanc, M., Zubek, R.: *MDA: A Formal Approach to Game Design and Game Research*. In: Proceedings of the AAAI Workshop on Challenges in Game AI, vol. 4, no. 1, p. 1722. San Jose, CA (2004)
24. Chen, J.: *Flow in Games (and Everything Else)*. Communications of the ACM, 50(4), 31–34 (2007)
25. Eyal, N.: *Hooked: How to Build Habit-Forming Products*. Penguin UK (2014)
26. Nielsen, J.: *Enhancing the Explanatory Power of Usability Heuristics*. In: Proceedings of the SIGCHI Conference on Human Factors in Computing Systems, pp. 152–158 (1994).